

CelineBitmap

OpenCV Image Laboratory · 이미지 처리와 픽셀아트 알고리즘 학습



OpenCV Image Laboratory

이미지를 읽고, 수치로 처리하고, 다시 눈에 보이는 결과로 만드는 연습.

C#과 Avalonia UI에서 OpenCvSharp5의 Mat, 색공간, 보간, 필터, K-Means와 Native 자원 수명주기를 직접 연결한 데스크톱 프로젝트입니다.

개발자 이준빈 (Celine)

SOLO

.NET 10

OPEN CV

2026.09

8

C# files

937

C# lines

15

commits

1.0.0

version

프로젝트 한눈에 보기

OpenCV 기본기를 실제 앱의 입력-처리-출력 흐름으로 정리

WHY

- OpenCVSharp 기본 구조 학습
- Unity 2D용 이미지 도구 준비
- RainAI 웹캠 비전의 전 단계

WHAT

- 리사이즈와 4종 보간
- 색상 조절과 6종 처리 함수
- K-Means 픽셀아트와 Alpha 보존

HOW

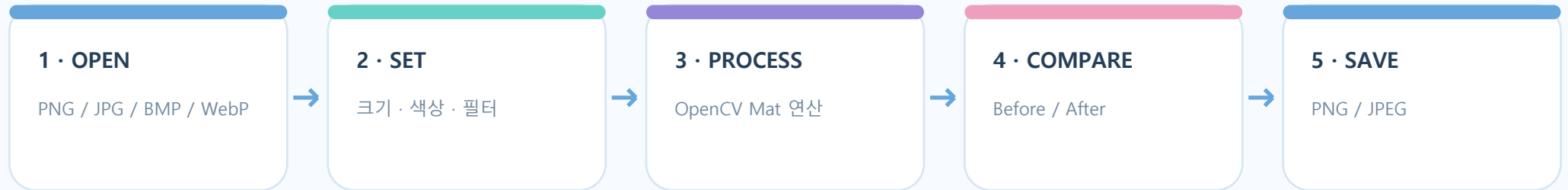
- Mat 기반 서비스 계층
- Before / After 즉시 비교
- 원본 재계산과 명시적 Dispose

현재 단계와 경계

구분	현재 상태	포트폴리오 표현
구현	이미지 I/O · Resize · Color · Filter · Pixel Art	소스 코드와 실제 생성 결과로 설명
검증	Release build · 예시 처리 · Alpha 경로	확인한 범위만 PASS로 표시
미구현	웹캠 · 얼굴 검출 · 얼굴 인식 학습	APC850/RainAI Roadmap으로 분리

사용 흐름

한 화면에서 설정을 바꾸고 Before / After를 비교한 뒤 저장



사용자 관점

- 이미지의 원본 크기를 확인하고 목표 해상도를 입력
- 비율 잠금 또는 25%~400% 버튼으로 빠르게 조정
- 보간법과 필터 차이를 Before / After로 즉시 확인
- 픽셀 Grid와 Palette 수를 선택해 픽셀아트화
- 작업 결과를 PNG 또는 JPEG로 저장

UI 구현 포인트

- Avalonia StorageProvider로 파일 선택과 저장
- NumericUpDown 값 변화에서 가로세로 비율 연동
- ToggleButton으로 여러 필터 조합
- 100ms debounce로 슬라이더 연속 입력 제어
- Mat과 WriteableBitmap 교체 시 이전 자원 해제

현재 기능 목록

UI 연결 여부와 서비스 구현 여부를 구분

영역	기능	UI	핵심 구현
Image I/O	열기 / 저장	연결	Cv2.ImRead · Cv2.ImWrite
Resize	크기·비율·배율	연결	Cv2.Resize
Interpolation	Lanczos4 · Cubic · Area · Nearest	연결	InterpolationFlags
Tone	밝기 · 대비 · 채도	연결	ConvertTo · BGR↔HSV
Filter	Gray · Blur · Sharpen · Edge	연결	CvtColor · GaussianBlur · Canny
Utility	Bilateral · Threshold	미연결	서비스 함수 구현
Pixel Art	Grid · Palette · Smooth · Alpha	연결	Area · Kmeans · Nearest · Merge

기술 스택과 선택 이유

UI, 이미지 처리, 상태 관리, 배포 기반을 작은 구조로 조합

C# / .NET 10

Native 라이브러리와 타입 안전한 서비스 코드를 연결

Avalonia UI 12.1.2

XAML 기반 Before / After 데스크톱 인터페이스

OpenCvSharp5

OpenCV 5의 Mat, imgproc, imgcodecs를 C#에서 호출

AvaloniaExtensions

Mat을 WriteableBitmap으로 변환해 UI에 표시

CommunityToolkit.Mvvm

확장 가능한 MVVM 의존성 기반

Git / GitHub

15개 커밋으로 기능 단위 변경 이력 관리

패키지 기준: OpenCvSharp5 5.0.0.20260905 · Avalonia 12.1.2 · CommunityToolkit.Mvvm 8.4.2

코드 구조와 데이터 흐름

View는 입력을 받고, Service는 Mat 연산을 담당

01

Avalonia View

MainWindow.axaml

이미지 미리보기 · 설정 컨트롤 · 파일 버튼

02

UI Orchestration

MainWindow.axaml.cs

상태 · 이벤트 · debounce · 수명주기

03

Image Services

ImageProcessor.cs

I/O · Resize · Tone · Filter · Alpha helper

04

Pixel Algorithm

PixelArtProcessor.cs

Bilateral · Area · K-Means · Nearest

05

Native Runtime

OpenCvSharp5 → OpenCV 5

Mat memory · imgproc · imgcodecs

핵심

OpenCV Mat은 Native 메모리를 소유하므로 계산 결과만큼 Dispose 흐름이 중요합니다.

Mat: 이미지가 숫자가 되는 방식

이미지는 Width × Height × Channel 형태의 행렬

PIXEL MATRIX

32	78	124	170	216
46	92	138	184	230
60	106	152	198	244
74	120	166	212	255

한 칸 = 한 픽셀 · 값 범위 0~255

MAT TYPE

예	의미
CV_8UC1	8-bit unsigned · Gray 1채널
CV_8UC3	8-bit unsigned · BGR 3채널
CV_8UC4	8-bit unsigned · BGRA 4채널
CV_32F	K-Means 입력에 사용하는 float

프로젝트에서의 Mat 수명

Load가 원본 Mat을 만들고, 각 처리 함수는 새 Mat을 반환합니다.

이전 결과를 교체할 때 Dispose하고, Window 종료 시 원본·작업·Bitmap을 모두 정리합니다.

BGR · BGRA · HSV · Gray

연산 목적에 맞는 색공간으로 변환한 뒤 필요한 채널만 조절

BGR

Blue · Green · Red

OpenCV 기본 컬러 순서

BGRA

BGR + Alpha

PNG 투명도까지 포함

HSV

Hue · Saturation · Value

채도 S만 독립 조절

GRAY

밝기 1채널

Canny와 Threshold 입력

채도 조정 흐름

SPLIT

BGR / Alpha



CONVERT

BGR → HSV



ADJUST

$S' = S \times \text{factor}$



RESTORE

HSV → BGR



MERGE

BGR + Alpha

Alpha를 색상 연산에서 분리하므로 채도 변경 뒤에도 원래 투명도를 다시 결합할 수 있습니다.

파일에서 화면까지

ImRead → Mat → WriteableBitmap → ImWrite

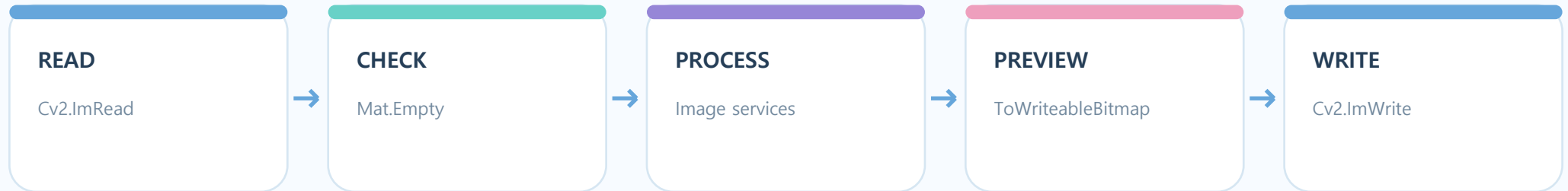


IMAGE LOAD

```

public static Mat Load(string path)
{
    Mat image = Cv2.ImRead(
        path, ImreadModes.Unchanged);

    if (image.Empty())
        throw new IOException(path);

    return image;
}
  
```

IMAGE SAVE

```

public static void Save(Mat image, string path)
{
    if (image.Empty())
        throw new ArgumentException();

    bool success = Cv2.ImWrite(path, image);
    if (!success)
        throw new IOException(path);
}
  
```

ImreadModes.Unchanged는 파일의 채널 구성을 가능한 그대로 읽어 PNG Alpha 보존의 출발점이 됩니다.

보간법 원리 표

확대·축소·픽셀아트마다 적절한 표본화 방식이 다름

Flag	권장 용도	계산 관점	결과 특징
Lanczos4	사진·일러스트 확대	8×8 주변 픽셀	선명한 결과를 기대할 수 있지만 계산량이 큼
Cubic	부드러운 확대	4×4 주변 픽셀	Bicubic 곡선으로 중간값을 추정해 계단을 완화
Area	이미지 축소	픽셀 면적 관계	다운샘플링 시 여러 원본 픽셀을 안정적으로 요약
Nearest	픽셀아트 확대	가장 가까운 1픽셀	새 색을 섞지 않아 사각형 도트 경계를 그대로 유지

선택 기준

- 일러스트를 2배 확대: Cubic 또는 Lanczos4를 비교
- 큰 이미지를 줄이기: Area
- 이미 만들어진 픽셀 Grid를 크게 보기: Nearest

Cubic 2×는 무엇을 하는가

출력 해상도는 2배, 정보량은 보간으로 추정

SIZE

$$W' = 2W$$

$$H' = 2H$$

픽셀 수는 4배가 되지만, 원본에 없던 실제 세부 정보를 새로 복원하는 것은 아닙니다.

BICUBIC

주변 4×4 픽셀

가로와 세로 방향의 곡선 가중치를 이용해
새 위치의 색을 부드럽게 계산합니다.

장점: 계단 완화 · 자연스러운 확대
주의: 미세한 흐림이나 halo가 생길 수 있음

EXACT 2× RESIZE

```
Cv2.Resize(
    source, result,
    new Size(source.Width * 2, source.Height * 2),
    0, 0, InterpolationFlags.Cubic);
```

밝기 · 대비 · 채도

산술 변환과 색공간 변환을 구분

BRIGHTNESS

$$\text{dst} = \text{src} + \beta$$

$\beta > 0$ 밝게
 $\beta < 0$ 어둡게

UI 범위: -100 ~ 100

CONTRAST

$$\text{dst} = \alpha \cdot \text{src}$$

$\alpha > 1$ 명암 증가
 $\alpha < 1$ 명암 감소

UI 범위: 0% ~ 200%

SATURATION

$$S' = S \times \text{factor}$$

BGR → HSV
 S 채널만 변경

Alpha 분리 후 재결합

코드 해석

함수	핵심 호출	현재 주의점
AdjustBrightnessContrast	Mat.ConvertTo(type, α , β)	BGRA 입력이면 Alpha까지 변환될 수 있음
AdjustSaturation	CvtColor + Split + ConvertTo + Merge	1채널은 Clone, 4채널은 Alpha 보존

Blur · Sharpen · Bilateral

주변 픽셀을 섞는 방식과 경계를 남기는 방식

함수	프로젝트 설정	원리	용도
Cv2.GaussianBlur	Size(5,5), sigma=0	가우시안 가중 평균	노이즈·세부를 부드럽게
Cv2.AddWeighted	$1.5 \times \text{src} - 0.5 \times \text{blur}$	흐린 성분을 빼 경계 강조	Unsharp masking 방식 샤프
Cv2.BilateralFilter	d=9, 75, 75 또는 d=5, 40, 40	거리와 색 차이를 함께 고려	평면은 매끈하게, 경계는 보존

GAUSSIAN → SHARPEN

blur = Gaussian(src)

result = 1.5 × src - 0.5 × blur

저주파 성분을 빼 상대적으로 경계를 강화

BILATERAL

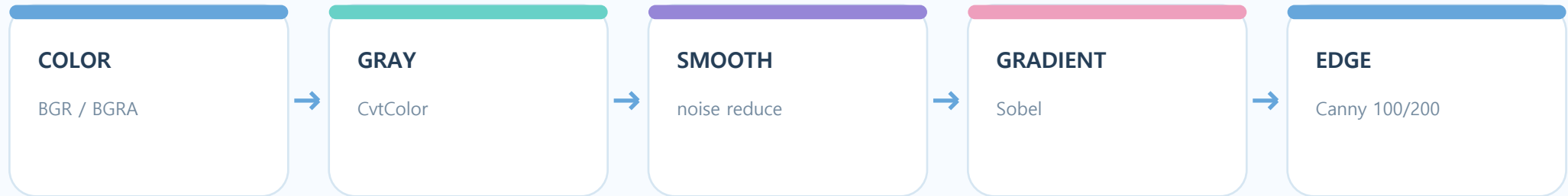
공간 거리 + 색상 거리

비슷한 색은 섞고 큰 색 차이는 덜 섞음

픽셀아트 전처리의 optional smoothing

Grayscale · Threshold · Canny

색상보다 밝기 변화와 경계 구조를 보는 연산



CANNY의 네 단계

- Gaussian smoothing으로 작은 노이즈 완화
- Sobel gradient로 밝기 변화의 크기와 방향 계산
- Non-maximum suppression으로 가는 경계만 남김
- 두 임계값 사이의 연결성을 이용해 최종 edge 결정

THRESHOLD

$\text{dst}(x,y) = 255 \text{ if gray} > T$

$\text{dst}(x,y) = 0 \quad \text{otherwise}$

프로젝트 기본 $T = 127$

서비스에는 구현되어 있으나
현재 UI 버튼에는 연결되지 않음

OpenCvSharp 함수 가이드

현재 코드에서 실제로 호출하는 핵심 API

함수	입출력	프로젝트에서의 역할
Cv2.ImRead / ImWrite	File ↔ Mat	이미지를 원래 채널로 읽고 처리 결과를 저장
Cv2.Resize	Mat → Mat	목표 크기와 InterpolationFlags로 재표본화
Cv2.CvtColor	Mat → Mat	BGR/BGRA/HSV/GRAY 사이의 색공간 변환
Cv2.Split / Merge	Mat ↔ channels	BGR, HSV, Alpha 채널을 분리하고 다시 결합
Cv2.GaussianBlur	Mat → Mat	가우시안 커널로 부드럽게 만들기
Cv2.AddWeighted	2 Mat → Mat	원본과 Blur를 가중합해 Sharpen
Cv2.Canny	Gray → Binary	강한 윤곽선을 0/255 영상으로 검출
Cv2.Threshold	Gray → Binary	기준값을 중심으로 완전한 흑백 분리

픽셀아트 관련 함수

색을 정돈하고, 줄이고, 군집화하고, 도트로 확대

함수/타입	핵심 매개변수	의미
Cv2.BilateralFilter	d · sigmaColor · sigmaSpace	경계를 보존하면서 내부 색을 정돈
Cv2.Resize + Area	pixelWidth × pixelHeight	원본을 저해상도 픽셀 Grid로 축소
Mat.Reshape	1 channel · pixelCount rows	H×W×3을 N×3 색상 점 목록으로 재배열
Mat.ConvertTo	CV_32F	K-Means가 사용할 float 샘플 행렬 생성
Cv2.Kmeans	K · criteria · attempts · PP centers	픽셀을 K개 대표 BGR 색상으로 군집화
AsRows<T>	Vec3b · int · float	행 단위 Span 접근으로 결과 픽셀을 채움
Cv2.Resize + Nearest	source.Size()	새 색을 섞지 않고 Grid를 원본 크기로 확대

픽셀아트 전체 파이프라인

Grid 해상도와 Palette 수를 독립적으로 제어

Pixel Art Pipeline



두 축 Pixel Width

형태를 표현하는 저해상도 Grid의 가로 칸 수

Palette Colors

K-Means가 만들 대표 색상 수: 8 / 16 / 32 / 64

Smooth 옵션

ON Bilateral로 색 영역을 먼저 정돈

OFF 원본 BGR을 그대로 Area 축소

목표 스타일과 원본 선명도에 따라 비교

K-Means 색상 양자화

각 픽셀을 [B, G, R] 좌표의 한 점으로 보기

CLUSTER VIEW



작은 점 = 원래 픽셀 · 큰 점 = centroid

ITERATION

1. K개의 초기 중심점을 선택
2. 각 픽셀을 가장 가까운 중심에 배정
3. 군집별 평균 BGR을 새 중심으로 계산
4. 이동량이 작거나 최대 반복에 도달할 때까지 반복
5. 모든 픽셀을 소속 군집의 대표 색으로 교체

목표 군집 내부 제곱거리 합을 작게 만들기

결과 수천 색 → K개의 일관된 Palette

주의: K는 샘플 픽셀 수 N보다 클 수 없으므로 초소형 Grid에는 $K \leq N$ 방어가 필요합니다.

K-Means 구현 코드

Reshape → CV_32F → labels/centers → Vec3b 출력

QUANTIZE K-MEANS

```
using Mat reshaped = source.Reshape(
    1, source.Rows * source.Cols);
using var samples = new Mat();
reshaped.ConvertTo(samples, MatType.CV_32F);

using var labels = new Mat();
using var centers = new Mat();
Cv2.Kmeans(samples, colors, labels,
    new TermCriteria(CriteriaTypes.Eps | CriteriaTypes.MaxIter, 20, 1.0),
    3, KMeansFlags.PpCenters, centers);

var outputRows = result.AsRows<Vec3b>();
var labelRows = labels.AsRows<int>();
var centerRows = centers.AsRows<float>();
```

자료구조

samples: $N \times 3$ float · **labels:** $N \times 1$ cluster index · **centers:** $K \times 3$ representative BGR

AsRows<T>()로 행 단위 Span을 사용해 각 출력 픽셀에 대표 색을 기록합니다.

투명도 보존 원리

색상 처리와 Alpha 처리를 분리한 뒤 같은 Grid에서 재결합

Alpha channel preservation in Pixel Art

투명 영역을 체크무늬 위에서 비교 — BGR 처리 후 Alpha를 같은 Grid로 재결합

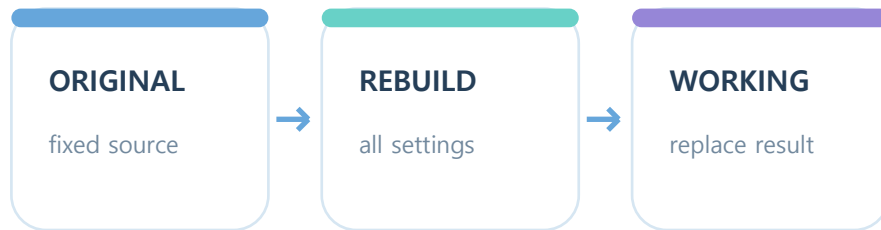


검증: 픽셀아트 출력은 BGRA 4채널을 유지했습니다. 체크무늬는 투명 영역을 보기 위한 배경입니다.

비파괴 재계산과 debounce

일반 조정은 항상 Original에서 다시 시작

NON-DESTRUCTIVE



같은 설정은 같은 결과를 만들고, 슬라이더 이동마다 이전 결과 위에 열화가 누적되지 않습니다.

100 ms DEBOUNCE



연속 입력 동안 예약을 갱신하고 마지막 값만 계산해 UI 부하를 줄입니다.

DEBOUNCE

```

sliderDebounceTimer.Interval = TimeSpan.FromMilliseconds(100);
sliderDebounceTimer.Tick += (_, _) =>
{
    sliderDebounceTimer.Stop();
    RebuildWorkingImage();
};
  
```

핵심 처리 코드

결과 Mat을 교체하면서 이전 Native 메모리를 즉시 해제

REBUILD WORKING IMAGE

```
Mat result = ImageProcessor.Resize(
    _originalImage, width, height, interpolation);

Mat next = ImageProcessor.AdjustBrightnessContrast(
    result, brightness, contrast);
result.Dispose();
result = next;

if (Math.Abs(saturation - 1.0) > 0.001)
{
    next = ImageProcessor.AdjustSaturation(result, saturation);
    result.Dispose();
    result = next;
}

_workingImage?.Dispose();
_workingImage = result;
```

장점 단계가 명시적이고, 각 함수가 새 Mat을 반환해 처리 순서를 읽기 쉽습니다.

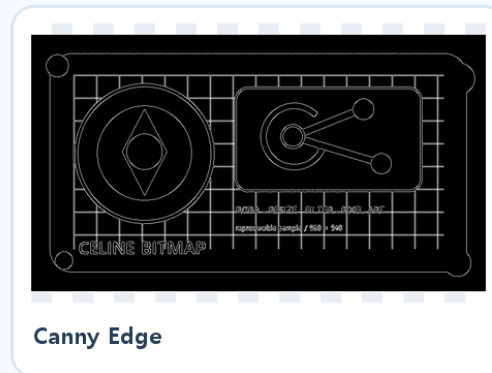
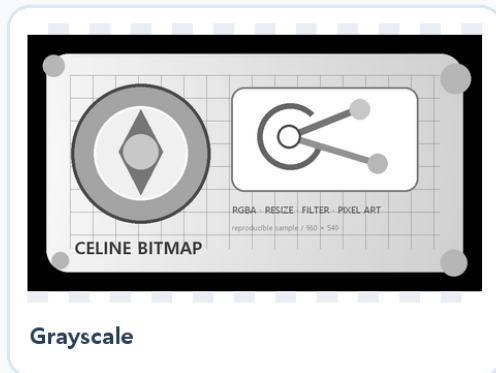
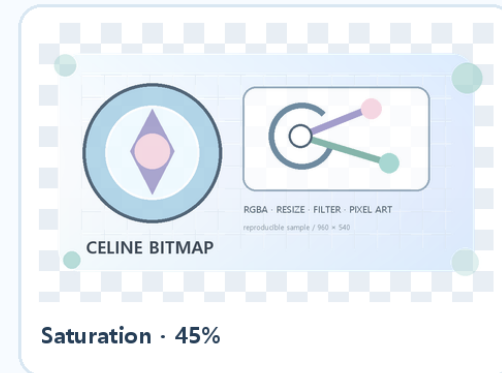
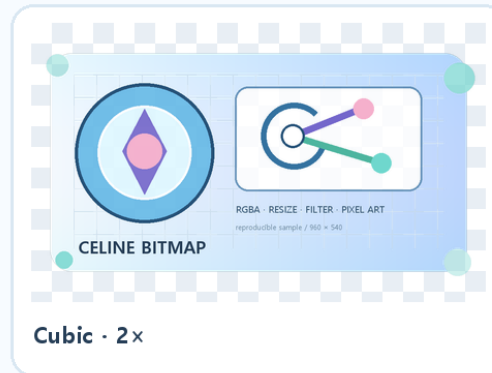
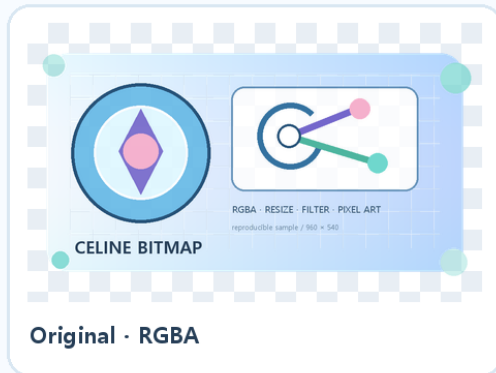
다음 Pixel Art도 동일한 상태 파이프라인에 포함하면 반복 적용과 Slider 재계산이 더 일관됩니다.

재현 가능한 결과 이미지

외부 게임 이미지를 사용하지 않고 자체 RGBA 테스트 카드에 현재 함수를 실행

CelineBitmap · reproducible processing samples

모든 결과는 아래 테스트 카드에 프로젝트의 현재 처리 함수를 적용해 생성



Original · Cubic 2x · Saturation 45% · Grayscale · Canny Edge · Pixel Art 72px/16 colors

검증 범위와 남은 개선

빌드 성공과 실제 예시는 확인했지만 전체 GUI 회귀 테스트는 별도

항목	상태	근거	의미
Release build	PASS	2026-09-16 · 1 warning / 0 errors	net10.0 Release 컴파일 성공
Processing samples	PASS	현재 서비스 코드로 6개 이미지 생성	I/O-Resize-Color-Gray-Edge-Pixel Art 실행
Alpha path	PASS	Pixel Art BGRA 4채널 결과	분리·Area/Nearest-Merge 경로 확인
GUI manual QA	NOT RUN	이번 PDF 수정에서는 미실행	모든 파일·버튼 조합의 사용자 시험은 아님
Automated tests	NONE	테스트 프로젝트 없음	경계값 회귀가 수동 확인에 의존

우선순위

순서	개선	이유
1	result?.Rows / Cols nullable 경고 제거	Release 경고 1개 정리
2	초소형 입력과 K > pixel count 방어	K-Means 예외 예방
3	밝기/대비에서 Alpha 분리 보존	BGRA의 투명도 일관성

ABKO APC850으로 넘어가기

정지 이미지 처리에서 실시간 프레임 처리로 확장

제조사 발표 사양

항목	APC850
Sensor	2 MP
Video	1920×1080 · 30 fps
Field of view	110°
Focus	넓은 고정 초점
Control	자동 밝기 · White Balance
Interface	USB · Plug and Play

비전 처리에 미치는 의미

- FHD 30fps 입력을 그대로 저장하기보다 추론 해상도와 주기를 분리
- 110° 화각은 주변 배경이 많이 들어오므로 얼굴 ROI를 먼저 검출
- 고정 초점은 카메라와 얼굴 사이의 학습 거리 범위를 미리 정해야 함
- 자동 밝기·White Balance 변화가 특징 벡터에 주는 영향을 여러 세션에서 측정
- 프라이버시 셔터와 앱의 Camera Off 상태를 일치시켜 사용자에게 명확히 표시

CAPTURE

1920×1080 / 30



DOWNSCALE

640×360 or 720p



INFER

every N frames



DISPLAY

latest result

위 해상도와 추론 주기는 측정 후 정할 목표값이며, 현재 구현 사실이 아닙니다.

실시간 얼굴 인식 파이프라인

Capture · Detect · Align · Embed · Compare · Stabilize



BASELINE: 사전학습 모델

- YuNet으로 얼굴 위치와 5개 landmark 검출
- SFace AlignCrop으로 얼굴 방향과 크기 정렬
- Feature로 고정 길이 embedding 생성
- 등록 embedding과 cosine similarity 또는 L2 비교
- 한 프레임이 아니라 최근 프레임 다수로 최종 판정

IMPORTANT

검출 ≠ 인식

검출은 얼굴이 어디 있는지 찾고,
인식은 그 얼굴이 누구인지 비교합니다.

모델 score가 높아도 신원 판정이 맞다는 뜻은 아닙니다. Detection threshold와 Identity threshold를 분리해야 합니다.

RainAI 기계학습 목표

사전학습 얼굴 모델을 기반으로 개인화된 Celine / Unknown 판정을 학습·보정

두 단계 목표

단계	목표
Baseline	YuNet + SFace 사전학습 모델과 등록 embedding 평균으로 먼저 동작
Personal ML	APC850로 모은 embedding에 개인 분류기/threshold calibration 적용
RainAI	신뢰도와 Unknown 상태를 대화·잠금·개인화 기능에 안전하게 연결

학습에서 확인할 질문

- 조명·안경·표정·거리 변화에서도 같은 사람으로 유지되는가?
- 다른 사람과 사진/화면 재생을 충분히 Unknown으로 거부하는가?
- 한 번의 오판이 아니라 시간축 confidence가 안정적인가?
- 등록 데이터를 지우면 모든 파생 embedding도 함께 삭제되는가?



제공 이미지: 레인 일러스트 (6).png · 문서 내 RainAI Roadmap 시각 자료로 사용

APC850 데이터 수집과 분할

같은 영상의 인접 프레임이 Train/Test에 섞이지 않도록 세션 단위로 분리

축	수집 예	왜 필요한가	주의
Lighting	낮 · 밤 · 역광 · 실내등	자동 밝기 변화 대응	조건별 세션을 따로 기록
Pose	정면 · 좌우 · 위아래	고개 각도 변화 대응	과도한 각도는 Unknown 허용
Appearance	안경 · 머리 · 표정	일상 변화 대응	민감 이미지 최소 보관
Distance	가까움 · 보통 · 멀리	고정 초점 조건 확인	권장 거리 범위를 UI에 안내
Negative	다른 사람 · 빈 화면 · 사진	Unknown / spoof 거부	동의 없는 타인 데이터 저장 금지

분할 원칙



성능 지표와 안전 기준

Accuracy 하나가 아니라 오인식과 미인식을 따로 측정

지표	질문	목표 해석
True Accept Rate	Celine을 Celine으로 인식하는가?	조명/각도/거리 조건별로 측정
False Accept Rate	다른 사람을 Celine으로 오인하는가?	가장 중요한 위험 지표 · 낮을수록 좋음
False Reject Rate	Celine을 Unknown으로 거부하는가?	불편함 지표 · 보안 수준과 함께 조정
Latency / FPS	화면과 판정이 충분히 빠른가?	Capture · Detect · Embed 시간을 각각 기록
Stability	프레임마다 결과가 흔들리지 않는가?	최근 N프레임 투표와 hysteresis로 완화

개인정보 원칙

- 명시적 등록 버튼과 Camera On 표시 · 로컬 우선 처리 · 원본 이미지 최소 보관
- 등록 취소 시 원본, crop, embedding, cache를 한 번에 삭제
- 애매한 점수는 사람 이름 대신 Unknown으로 반환

CelineBitmap에서 RainAI까지

이미지 처리 기초를 실시간 비전과 개인화 머신러닝으로 확장

DONE

CelineBitmap

Mat · Resize · Color · Filter · K-Means · Alpha

STEP 1

APC850 Capture

VideoCapture · frame rate · camera lifecycle

STEP 2

Face Baseline

YuNet · SFace · embedding · Unknown

STEP 3

Personal ML

session dataset · calibration · evaluation

STEP 4

RainAI Integration

confidence · temporal state · safe action

완료 기준: 데모가 켜지는 것보다, 다른 사람을 Unknown으로 거부한 측정 로그와 재현 가능한 테스트가 남는 것.

근거 파일과 공식 참고 문서

구현 사실은 저장소 소스, 하드웨어·알고리즘 설명은 공식 자료를 기준으로 작성

PROJECT EVIDENCE

파일	확인 내용
CelineBitmap.csproj	.NET 10 · Avalonia 12.1.2 · OpenCvSharp5 5.0.0.20260905
ImageProcessor.cs	I/O · Resize · Tone · Filter · Split/Merge · Save
PixelArtProcessor.cs	Bilateral · Area · K-Means · Nearest · Alpha reconstruction
MainWindow.axaml(.cs)	Before/After UI · state · debounce · disposal

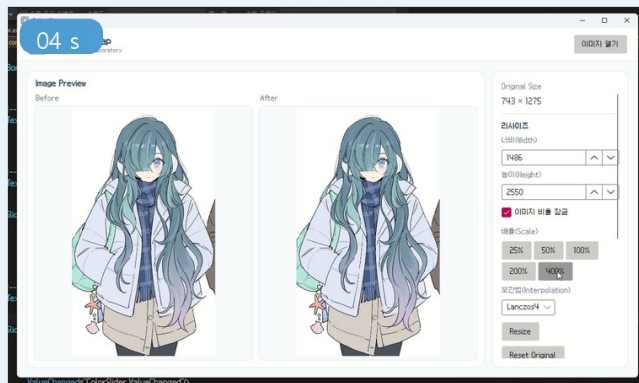
OFFICIAL REFERENCES

- [ABKO · APC850 출시 및 공식 사양](#)
- [OpenCV · K-Means Clustering](#)
- [OpenCV · DNN Face Detection and Recognition](#)
- [OpenCvSharp · official repository](#)
- [OpenCV · Geometric Image Transformations](#)
- [OpenCV · Canny Edge Detection](#)
- [OpenCV · VideoCapture](#)
- [Avalonia · documentation](#)

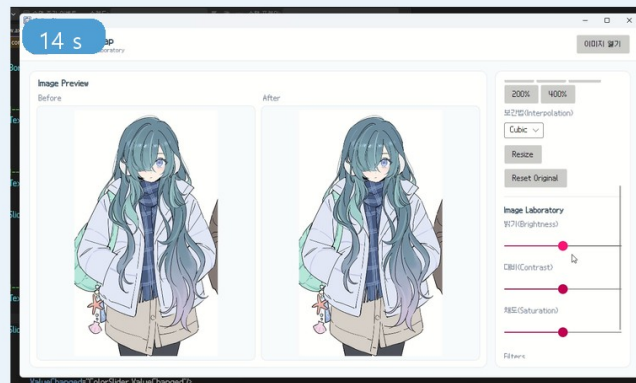
40초 실행 영상으로 확인한 실제 동작

정지 이미지뿐 아니라 조작 순서와 결과 변화가 이어지는 화면 기록

VIDEO CONTACT SHEET



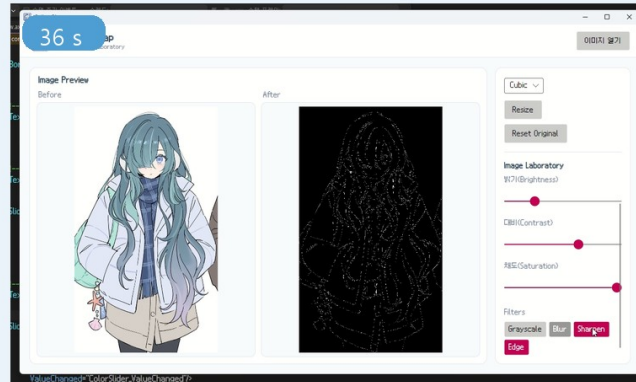
Resize preset and ratio lock



Brightness / contrast / saturation



Tone adjustment preview



Canny edge result

원본: CelineBitmap OpenCV 테스트1.mp4 · 대표 시점 4 / 14 / 26 / 36초

영상 정보

40.00 s

1320 × 784 · 30 fps

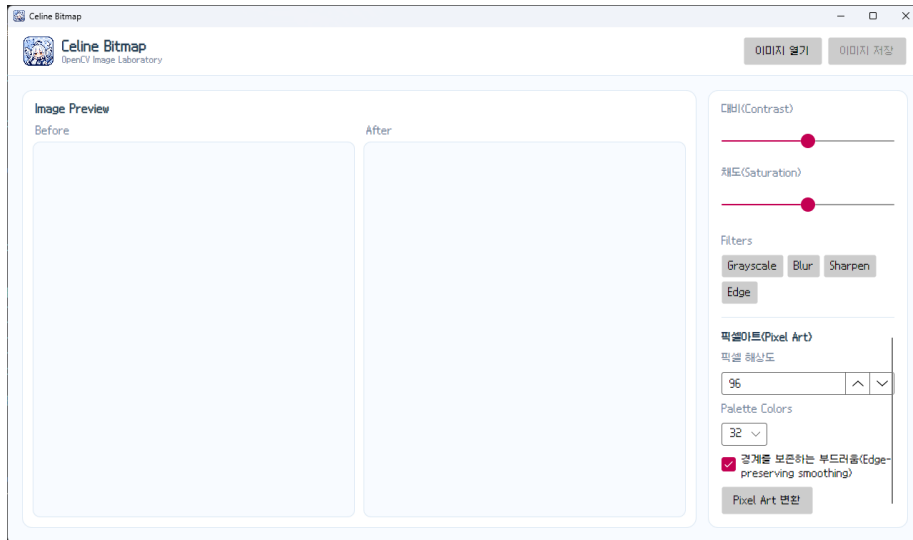
확인된 흐름

- 원본 이미지 열기
- 비율 잠금과 배율 변경
- Cubic 보간 리사이즈
- 밝기·대비·채도 조정
- Canny Edge 적용
- Before / After 비교

실제 UI와 리사이즈 사용 사례

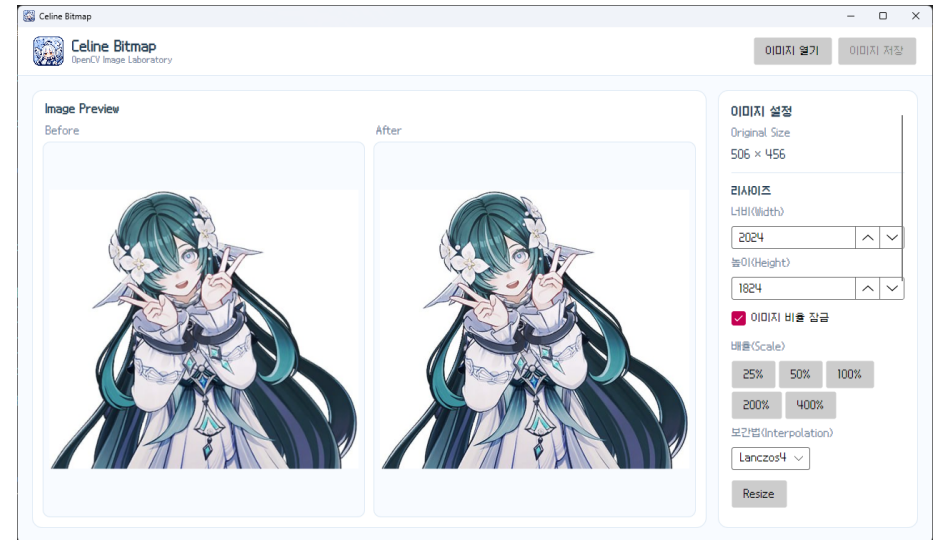
기능 목록이 아니라 사용자가 보는 Before / After 작업 흐름을 제시

초기 작업 화면



CelineBitmap UI.png · 이미지 로드 전 상태

Lanczos4 400% 리사이즈



리사이즈 예시_보다니차.png · 506×456 → 2024×1824

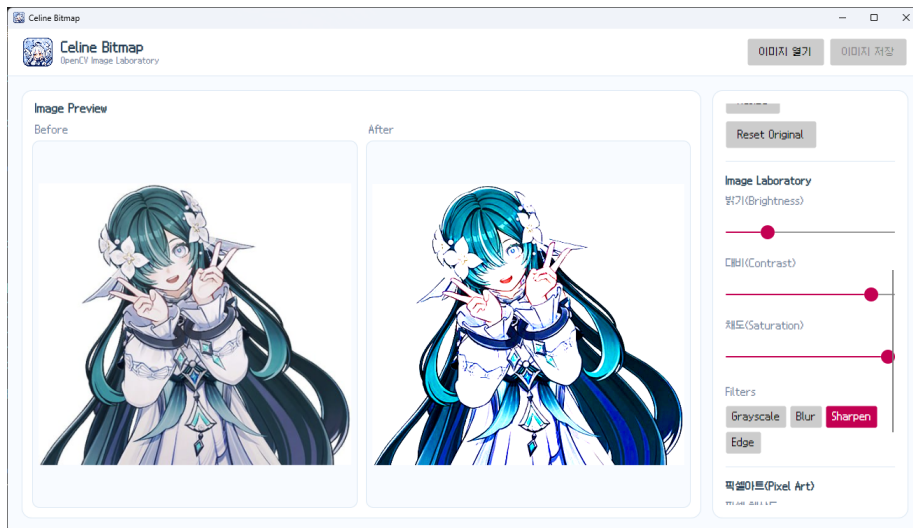
이 화면이 보여주는 것

- 왼쪽 Before는 원본 확인, 오른쪽 After는 처리 결과 확인에 집중합니다.
- 비율 잠금, 배율 프리셋, 보간법 선택을 한 패널에서 조작합니다.
- Lanczos4 사례는 확대 배율과 최종 픽셀 크기를 동시에 기록합니다.

톤 조절과 픽셀아트 변환 실험

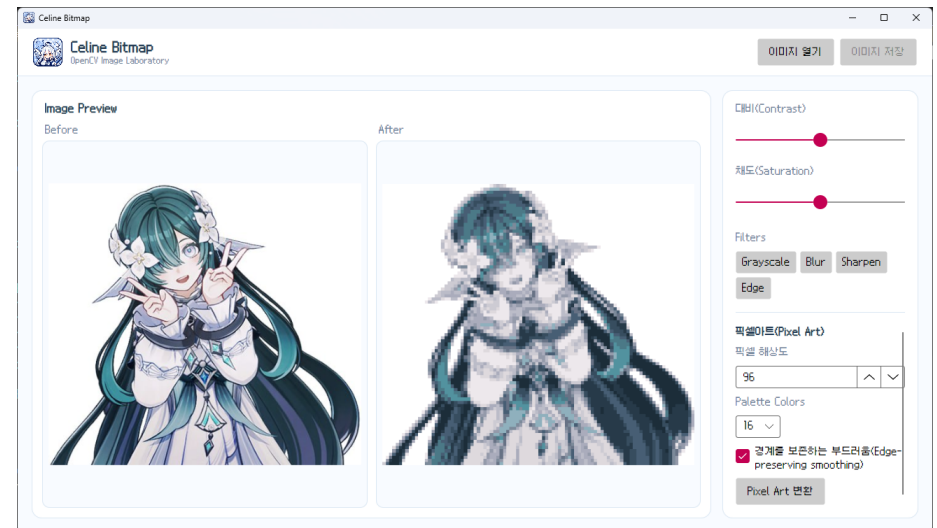
같은 입력 이미지에서 연속 톤 처리와 K-Means 픽셀화를 비교

밝기·대비·채도 + Sharpen



이미지 HSV 등 변형_보다니차.png

96px Grid · 16 Colors · Smoothing



픽셀아트화_보다니차.png

해석

톤 조절

HSV의 S 채널과 ConvertTo 기반 밝기·대비가 선명도 필터와 결합된 예시

픽셀아트

Area 축소 → K=16 양자화 → Nearest 확대의 결과를 실제 UI에서 확인

수치가 큰 톤 조절은 흰색 clipping과 색 경계를 강조할 수 있으므로, 결과를 보며 목적에 맞게 조절해야 합니다.

물의 구체를 픽셀 리소스로 바꾸는 과정

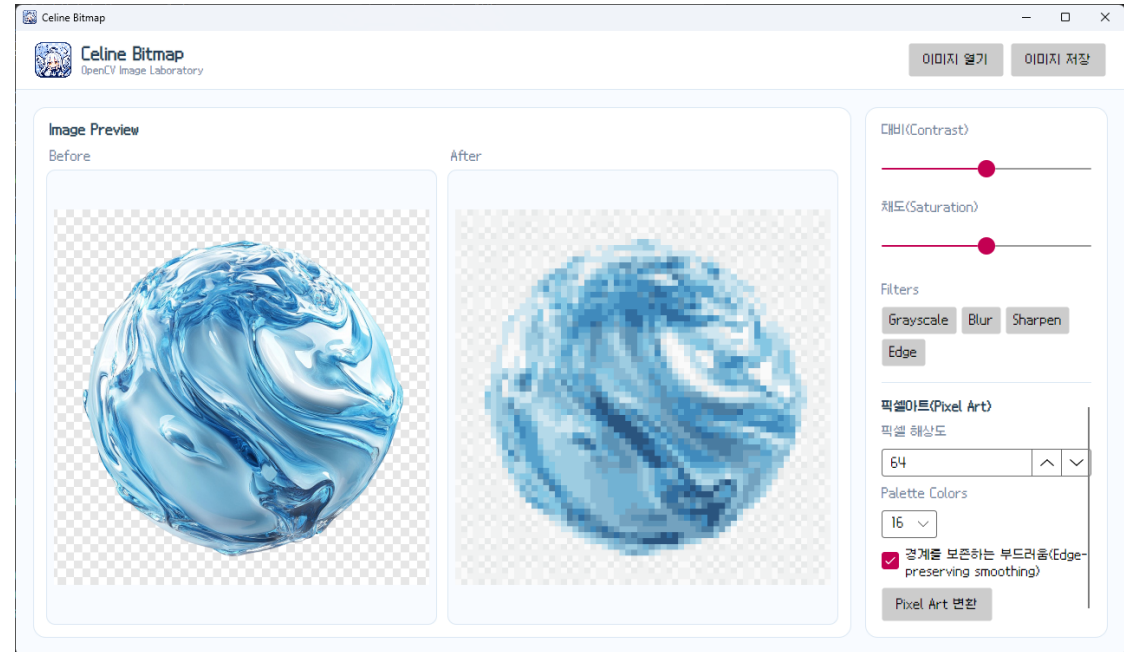
실제 Unity 2D 게임 리소스 제작을 염두에 둔 입력 → 변환 사례

SOURCE



WaterOrb_example.jpg · 원본의 체크무늬 포함

CELINEBITMAP WORKFLOW



64px Grid · 16 Colors · 경계 보존 Smoothing

INPUT

water orb



GRID

64 px



PALETTE

K = 16



UPSCALE

Nearest



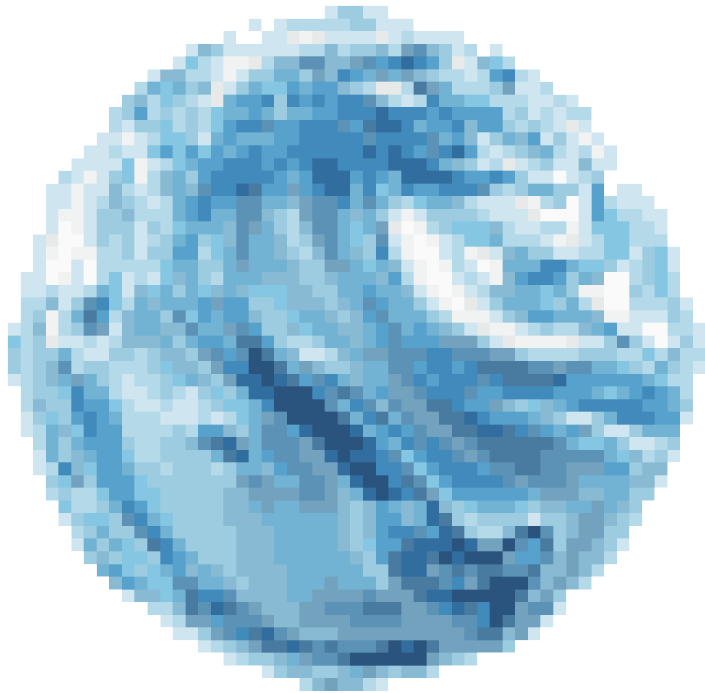
SAVE

game asset

저장 결과와 Unity импорт 체크리스트

픽셀화가 끝난 뒤에도 게임 엔진에서 도트 경계와 투명도를 다시 확인

SAVED OUTPUT



CelineBitmap_Output.png · 실제 저장 결과

Unity 2D에서 확인할 항목

- Texture Type: Sprite (2D and UI)
- Filter Mode: Point - 도트 경계 흐림 방지
- Compression: 품질 비교 단계에서는 None 권장
- Pixels Per Unit: 캐릭터·타일 기준 Grid에 맞춤
- Mip Map: 2D UI/근거리 스프라이트 목적에 맞게 결정
- Alpha Is Transparency와 가장자리 halo를 실제 배경에서 확인

주의

이 출력 예시의 바깥 영역은 흰색으로 보입니다. 실제 투명 PNG 여부는 Alpha 채널 검사와 Unity 배경 합성으로 별도 검증해야 합니다.

캐릭터·마법 리소스로 확장할 다음 실험

물·땅 속성처럼 색과 발광이 다른 원화에서 설정별 결과를 축적

SOURCE REFERENCE



땅의 마법사 카논 테스트.png · 확장 테스트용 입력 이미지

실험 설계

- Resize: 캐릭터 초상화·대화창·아이콘 규격별 크기 비교
- HSV: 주황 발광의 채도와 밝기 clipping 구간 기록
- K-Means: 8 / 16 / 32 / 64색으로 금색·갈색 계조 보존 비교
- Smoothing: 머리카락·지팡이 외곽선 보존 효과 비교
- Canny: 마법진과 결정 형태를 분리하는 threshold 실험
- Unity: Point 필터와 실제 배경에서 가독성·halo 확인

구현 사실과 계획의 구분

현재 이미지는 처리 결과가 아닌 테스트 입력 후보입니다. 위 항목은 CelineBitmap의 기존 기능으로 이어갈 다음 검증 계획입니다.